

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	C Shahanaj	Reg. No.:	192372292
Section / Batch:	CSE AI	Date:	16/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

Solving Task		
---------------------	--	--

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____shahanaj / 192372292_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____shahanaj_____

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Code:

```
def solve():
```

```
    data = list(map(float, input().split()))
```

```
    n = int(data[0])
```

```
    demand = list(map(int, data[1:n + 1]))
```

```
    supply = list(map(int, data[n + 1:2 * n + 1]))
```

```
    peak = int(data[2 * n + 1])
```

```
    # Optional energy-loss rate
```

```
    if len(data) > 2 * n + 2:
```

```
        rate = data[2 * n + 2]
```

```
    else:
```

```
        rate = 1.0
```

```
    # DP[t][energy] = minimum loss
```

```
    total_demand = sum(demand)
```

```
    INF = float('inf')
```

```
    dp = [INF] * (total_demand + 1)
```

```
    dp[0] = 0
```

```
    for t in range(n):
```

```
        new_dp = [INF] * (total_demand + 1)
```

```
        capacity = min(supply[t], peak)
```

```
        for used in range(total_demand + 1):
```

```

if dp[used] == INF:
    continue

# Amount of electricity distributed
for amount in range(capacity + 1):
    if used + amount > total_demand:
        break

# Cannot distribute more than current demand
if amount > demand[t]:
    continue

loss = (supply[t] - amount) * rate

new_value = dp[used] + loss

if new_value < new_dp[used + amount]:
    new_dp[used + amount] = new_value

dp = new_dp

# Check if total demand can be satisfied
if dp[total_demand] == INF:
    print("1000000000000000000.000000")
else:
    print(f"{dp[total_demand]:.6f}")

```

solve()

output:

ASSESSMENT C04-AT2-Industry Problem Solving Task

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Smart Grid Energy Distribution Optimization (Dynamic Programming)

100 marks

1/1 answered

Q1 Smart Grid Energy Distribution Optimization (Dynamic Programming)

100 marks

A smart grid system must distribute electricity efficiently to minimize energy loss and meet varying demand. Analyze how optimization algorithms can be applied to this problem. Identify constraints such as peak load and limited supply. Design a solution using dynamic programming or greedy approaches. Discuss scalability in large energy networks. Evaluate efficiency and computational complexity. Interpret results in sustainable energy systems.

Your Code

```
1 import sys
2 def solve():
3     data = sys.stdin.read().strip().split()
4     if not data:
5         return
6     idx = 0
7     T = int(data[idx]); idx += 1
8     D = list(map(float, data[idx:idx+T])); idx += T
9     S = list(map(float, data[idx:idx+T])); idx += T
10    peak = float(data[idx]); idx += 1
11    rate = 1.0
12    if idx < len(data):
13        rate = float(data[idx]); idx += 1 # optional last input
14    step = 1.0
15    def to_units(x):
16        return int(round(x / step))
17
18    D_u = [to_units(x) for x in D]
19    S_u = [to_units(x) for x in S]
20    peak_u = to_units(peak)
21
22    max_units = max(S_u + D_u)
23    INF = 10**18
24    dp_prev = [INF] * (sum(D_u) + 1)
25    dp_prev[0] = 0.0
26    total_demand = sum(D_u)
27    for t in range(T):
28        dp_curr = [INF] * (total_demand + 1)
29        supply_cap = min(S_u[t], peak_u)
30        for delivered_before in range(total_demand + 1):
31            if dp_prev[delivered_before] >= INF:
32                continue
33            remaining = total_demand - delivered_before
```

Input

```
4
25 25 25 25
15 20 10 5
22 22 22 22
```

Output

```
10000000000000000.000000
```